

Grok Build Development Guide

Canonical architecture and local-development reference (source: GROK.md)

Generated 2026-09-25

Contents

1	Grok Build Development Guide	1
1.1	Language	2
1.2	What This Project Is	2
1.3	Repository Map	2
1.3.1	Product Core	2
1.3.2	Supporting Systems	2
1.4	Runtime Architecture	3
1.4.1	TUI	3
1.4.2	Agent Runtime and Sessions	3
1.4.3	Tools	3
1.4.4	Permissions and Sandbox	4
1.4.5	Retrieval and Prime	4
1.4.6	Token Conservation (Tersify)	4
1.4.7	MCP	5
1.4.8	Leader Mode	5
1.5	Sealed Development Console	5
1.5.1	Non-Negotiable Profile	5
1.5.2	Canonical Environment	5
1.5.3	Required Preflight	6
1.5.4	Forbidden Profiles and Artifacts	6
1.6	Development Commands	6
1.6.1	Rapid development flow	7
1.6.2	One helper at a time	7
1.6.3	Helper scripts	7
1.6.4	Direct Cargo commands	8
1.6.5	Test runners	8
1.6.6	Running the console	8
1.6.7	Interactive TUI verification with tmux	9
1.6.8	Headless mode	9
1.6.9	Makefile (release-dist and deployment only)	9
1.6.10	Version identity and bumps (fcustom)	10
1.6.11	Cargo.lock discipline	11
1.6.12	Tooling policy	11
1.6.13	Measuring compile/link cost	11
1.7	Testing Strategy	12
1.8	Change Guidelines	12
1.9	Important Starting Points	12
1.10	Definition of Done	12

1 Grok Build Development Guide

This document is the canonical technical and local-development reference for this checkout. It explains the "why" and "how" behind the mandatory rules in [AGENTS.md](#). Repository agents must read both files before

making changes.

- [AGENTS.md](#) is the enforceable operations policy: what agents must and must not do.
- [GROK.md](#) (this file) is the detailed canonical architecture and development guide.
- [CLAUDE.md](#) is a concise compatibility entry point for Claude-compatible agents; it still requires reading AGENTS.md and GROK.md in full before editing.

1.1 Language

All new project communication and authored content must use United States English (en-US). This includes documentation, code comments, test names, diagnostics, commit messages, and user-facing strings. Existing content should not be translated as part of an unrelated change.

1.2 What This Project Is

Grok Build is a terminal-based AI coding agent implemented as a Rust 2024 workspace. It supports:

- an interactive full-screen terminal UI;
- single-turn and headless automation;
- stdio integration through the Agent Client Protocol;
- shared leader processes for reconnecting and multi-client sessions;
- model inference and tool calling;
- filesystem, shell, VCS, search, MCP, and media tools;
- session persistence, worktrees, checkpoints, memory, and compaction;
- permission policies and operating-system sandboxing;
- retrieval and Prime (skill/agent ranking, metadata index, strict skills);
- token conservation (tersify) and a streaming repetition-loop guard.

The composition-root package is `xai-grok-pager-bin`. It builds the `xai-grok-pager` artifact, which official releases expose as `grok`.

This fork runs without an xAI account; [docs/xai-decoupling.md](#) is the full story (every surface, every key, the measurements, and the reproducible battery in [grok-off-xai-check.sh](#)).

1.3 Repository Map

1.3.1 Product Core

Path	Responsibility
<code>crates/codegen/xai-grok-pager-bin</code>	Process startup, CLI dispatch, crash handling, sandbox startup, leader connection
<code>crates/codegen/xai-grok-pager</code>	TUI state, input, rendering, scrollback, commands, and user interaction
<code>crates/codegen/xai-grok-shell</code>	Agent runtime, sessions, inference, tools bridge, stdio/headless/serve/leader mode
<code>crates/codegen/xai-grok-tools</code>	Built-in tool descriptions, registry, dispatch, persistence, and implementations
<code>crates/codegen/xai-grok-workspace</code>	Filesystem and VCS abstraction, execution, permissions, checkpoints, worktrees

1.3.2 Supporting Systems

Path	Responsibility
<code>crates/codegen/xai-grok-config</code>	Layered configuration, managed requirements, Grok home resolution, etc.
<code>crates/codegen/xai-grok-config-types</code>	Shared configuration schema
<code>crates/codegen/xai-grok-sandbox</code>	Workspace, strict, devbox, custom, and disabled sandbox profiles
<code>crates/codegen/xai-grok-mcp</code>	MCP server transports, credentials, OAuth, liveness, and ACP bridging
<code>crates/codegen/xai-grok-agent</code>	Agent prompt templates and skill-related prompt construction
<code>crates/codegen/xai-grok-tersify</code>	Token-saving engine: safety ladder, compression, retrieval handles, and more
<code>crates/codegen/xai-grok-memory</code>	Cross-session memory, indexing, search, and consolidation
<code>crates/codegen/xai-grok-pager-render</code>	Themes, syntax presentation, terminal glyphs, and rendering primitives
<code>crates/codegen/xai-grok-pager-ptty-harness</code>	PTY-based test infrastructure
<code>crates/common/xai-tool-runtime</code>	Core Tool, ToolDyn, streaming, and typed-output abstractions
<code>crates/common/xai-tool-protocol</code>	JSON-RPC protocol frames and tool/session identifiers

Path	Responsibility
crates/common/xai-computer-hub-*	Tool-server registry, transport, pooling, and MCP adaptation
crates/common/xai-grok-compaction	History compaction strategies and host-independent traits
third_party	Vendored Mermaid rendering stack

The root `Cargo.toml` is generated. Treat it as read-only and edit the relevant crate manifest instead.

1.4 Runtime Architecture

The primary executable entry point is:

```
crates/codegen/xai-grok-pager-bin/src/main.rs
```

Startup follows this general sequence:

```
main
-> early subprocess dispatch and CLI parsing
-> crash, telemetry, and runtime setup
-> managed requirements validation
-> GROK_HOME-backed configuration loading
-> sandbox profile resolution and installation
-> command dispatch
    -> interactive TUI
    -> single-turn/headless
    -> ACP stdio
    -> WebSocket serve
    -> shared leader
-> agent session
-> model inference
-> tool selection and dispatch
-> permission gate
-> workspace or external operation
-> persisted session and UI update
```

1.4.1 TUI

The TUI is centered in `xai-grok-pager/src/app`. It uses an Elm-like separation:

- actions describe events;
- dispatch mutates synchronous state and produces effects;
- effects perform asynchronous work;
- `AppView` is the root input and drawing component;
- the event loop coordinates terminal, agent, and task events.

Visible TUI changes should be tested through render assertions, snapshots, scripted scenarios, or the PTY harness.

1.4.2 Agent Runtime and Sessions

`xai-grok-shell` owns agent modes and durable session behavior. Sessions are ACP-oriented and coordinate prompts, model turns, tool calls, persistence, compaction, goals, memory, MCP servers, and worktrees.

Session state is stored beneath:

```
$GROK_HOME/sessions/
```

This is one reason development must never use the default `~/.grok` profile.

1.4.3 Tools

`xai-grok-tools` registers built-in tools and adapts them to the shared tool runtime. Tool execution can stream progress before producing one terminal result. Filesystem, shell, web, MCP, LSP, media, task, and

editing tools share common session resources.

Tool changes must preserve:

- typed argument validation;
- permission classification and prompting;
- progress and terminal-result ordering;
- path and workspace containment;
- durable state and notification behavior;
- safe error reporting without credential leakage.

1.4.4 Permissions and Sandbox

Permissions and sandboxing are distinct layers:

- permission policy decides whether an operation is allowed, denied, or must be confirmed;
- sandboxing constrains what the operating system permits the process or child process to access.

Do not treat one as a replacement for the other. Changes to shell parsing, command wrappers, path matching, MCP access, web access, or sandbox profiles are security-sensitive and require adversarial failure-path tests.

1.4.5 Retrieval and Prime

Prime selects a small set of skills (and advisory agents) for the next turn from a strict inventory. Ranking fuses local/path evidence with optional BM25 and sqlite-vec KNN over a disposable metadata index. Vectors are locally L2-normalized (l2_v1) and fingerprinted so mixed embedding spaces cannot share a collection.

SQLite remains the authoritative vector store for this path. For memory, [memory] mode selects between "local" (the default: local SQLite index with sqlite-vec and FTS5, zero Milvus involvement) and "milvus" (hard-remote primary store: Milvus BM25 keyword search and dense KNN, local SQLite for chunk book-keeping only). The mode is selectable in the TUI (/retrieval-settings) and configurable per-workspace in .grok/config.toml (gated by folder trust). A [vector_stores.<id>] entry (backend milvus) describes the remote Milvus server: mirror reads require a fingerprint/dims/row-count match and fall back to sqlite-vec in local mode, while milvus mode enforces hard-remote semantics with no silent fallback. Resync streams vectors from SQLite without re-embedding. The mirror bearer token resolves from the vault scope milvus::<store-id>::token or the MILVUS_TOKEN_FOR_<ID> environment variable, never from config. A configured store receives memory text, so mirror tests stay hermetic (fake mirrors); live-server conformance tests are #[ignore]-gated.

Development and tests for this path must stay hermetic: no live embedding or rerank providers, no production GROK_HOME, and no secret/path leakage in inspect JSON, logs, or rendered <skill_prime> blocks. Routing-quality metrics live beside the Prime code. Operator docs:

- [crates/codegen/xai-grok-pager/docs/user-guide/13-memory.md](#)
- [crates/codegen/xai-grok-pager/docs/user-guide/30-retrieval-and-prime.md](#)
- [crates/codegen/xai-grok-pager/docs/user-guide/31-strict-skills-migration.md](#)

1.4.6 Token Conservation (Tersify)

crates/codegen/xai-grok-tersify is the token-saving engine; xai-grok-shell integrates it into the session runtime.

Engine responsibilities:

- safety ladder S0-S4 as an exhaustive match, gating what a compressor may claim (coverage is never overstated);
- deterministic token counting through the Counter trait (default approx-chars/4, rune-based);
- payload routing (detect) across JSON, terminal, diff, code, log, and search shapes, failing open to plain text;
- compressors: marker (asserts only verified facts; credentials and free-form text are retained whole) and log (S4: signal/edge lines and boundaries survive; identical runs collapse to a representative plus a marker);
- a persistent SQLite store (WAL, size budget; on refusal the engine passes the original through) with content-addressed rcv_<handle> retrieval pointers;
- record mode is fail-closed: engine failures never emit compressed bytes.

Integration points:

- the spawn gate (`build_spawn_system_prompt`) appends `<tersify_style>` only where the configured scope allows it (`main_only` default, `all`, or `off`); subagent outputs stay raw by default;
- `/tersify lite|full|ultra|off` sets a session-only level through `PromptRequest._meta.tersifyLevel` (`SetSessionTersifyLevel`), and the style block is restamped each turn (`refresh_tersify_style_impl` via `ReplaceSystemHead`);
- tool results larger than 4 KiB route through `push_tool_result_tersified` with `Tersify::transform_fn()`; error results are never compressed and inline images are untouched.

The streaming repetition-loop guard (`xai-grok-shell/src/session/repetition_guard.rs`) consumes each Text token and detects non-alphanumeric character floods and n-gram loops; on detection it surfaces a visible warning, latches, and cancels the turn. It is on by default and toggleable via the `repetition_guard` setting (persisted under `[hints]`).

1.4.7 MCP

`xai-grok-mcp` owns MCP transports, authentication, OAuth, process lifecycle, server health, and tool invocation. MCP credentials are stored below `GROK_HOME`; they must remain separate from production credentials.

1.4.8 Leader Mode

Leader mode allows multiple clients to share an agent process and supports reconnection with ACP state replay. A leader can outlive the binary invocation that started a client, so accidentally connecting a new build to an old leader can invalidate a test.

Ordinary build verification must use `--no-leader`. Tests specifically covering leader behavior must use the isolated development socket:

```
~/ .grokdev/leader.sock
```

1.5 Sealed Development Console

1.5.1 Non-Negotiable Profile

All repository builds, tests, inspections, and executions must use:

```
~/ .grokdev
```

The application reads the location from `GROK_HOME`. Use the absolute, shell-expanded form:

```
export GROK_HOME="${HOME}/.grokdev"
```

The `--agent-profile` CLI option does not select this profile. It points to an agent-definition file. Development-home isolation must always use `GROK_HOME`.

Do not pass the literal string `~/ .grokdev` through an API that does not perform shell expansion.

`GROK_HOME` affects configuration, authentication, sessions, memory, logs, MCP credentials, marketplace data, installed Grok paths, and other persistent runtime state. The application caches this path, so it must be set before the process starts.

1.5.2 Canonical Environment

Run development commands in a shell configured like this:

```
set -euo pipefail
umask 077

export GROK_HOME="${HOME}/.grokdev"
export GROK_LEADER_SOCKET="${GROK_HOME}/leader.sock"
export GROK_DISABLE_AUTOUPDATER=1

export GROK_CURSOR_SKILLS_ENABLED=0
export GROK_CURSOR_RULES_ENABLED=0
```

```
export GROK_CURSOR_AGENTS_ENABLED=0
export GROK_CURSOR_MCPS_ENABLED=0
export GROK_CURSOR_HOOKS_ENABLED=0
export GROK_CURSOR_SESSIONS_ENABLED=0

export GROK_CLAUDE_SKILLS_ENABLED=0
export GROK_CLAUDE_RULES_ENABLED=0
export GROK_CLAUDE_AGENTS_ENABLED=0
export GROK_CLAUDE_MCPS_ENABLED=0
export GROK_CLAUDE_HOOKS_ENABLED=0
export GROK_CLAUDE_SESSIONS_ENABLED=0

export GROK_CODEX_SKILLS_ENABLED=0
export GROK_CODEX_RULES_ENABLED=0
export GROK_CODEX_AGENTS_ENABLED=0
export GROK_CODEX_MCPS_ENABLED=0
export GROK_CODEX_HOOKS_ENABLED=0
export GROK_CODEX_SESSIONS_ENABLED=0

mkdir -p "${GROK_HOME}"
chmod 0700 "${GROK_HOME}"
```

This environment prevents the development console from sharing the normal Grok profile, leader socket, auto-updater, or compatibility resources belonging to Cursor, Claude, or Codex.

Compatibility integration tests may explicitly enable the exact compatibility surface they exercise. They must use controlled fixtures and must not inspect or modify real external-agent state.

1.5.3 Required Preflight

Before launching repository code, verify:

```
test "${GROK_HOME}" = "${HOME}/.grokdev"
test "${GROK_LEADER_SOCKET}" = "${HOME}/.grokdev/leader.sock"
test -d "${GROK_HOME}"
```

If any check fails, stop. Do not fall back to `~/.grok`.

1.5.4 Forbidden Profiles and Artifacts

Development must not access or mutate:

- `~/.grok`;
- `~/.grok-prod`;
- another user's Grok home;
- `/opt/grok-custom/grok`;
- the installed grok executable;
- the grok-custom deployment wrapper.

Do not run `make deploy`, `make deploy-binary`, `make deploy-wrapper`, or `make verify` during normal development. Those targets install signed artifacts outside the repository and are reserved for explicit deployment work.

Do not copy authentication or configuration from another profile. Authenticate the development profile independently when necessary.

1.6 Development Commands

Every example below must run in the canonical environment from the previous section, in the same shell invocation.

1.6.1 Rapid development flow

The normal edit loop is intentionally narrow and fast. Use the helper scripts that share one worktree-local `./target-dev` directory:

1. **Fast focused check.** `./grok-check.sh -p <crate>` compiles the crate and its immediate dependencies under `./target-dev`. It is the fastest signal for type errors and is the first command after a small edit.
2. **Focused test.** `./grok-test.sh -p <crate>` runs the crate's tests through `cargo-nextest`, reusing the same `./target-dev` artifacts.
3. **Validate consumers.** After the affected crate passes, check and test the direct consumers that exercise the changed surface.
4. **Broader validation** only when the change crosses enough crate boundaries to justify the cost.
5. **Release-dist or make build** only as a deliberate final step, not inside the ordinary loop.

```
# 1. Affected crate.
./grok-check.sh -p xai-grok-shell
./grok-test.sh -p xai-grok-shell
./grok-test.sh -p xai-grok-shell -- session_runtime_family

# 2. Direct consumer. Check that the composition-root binary still compiles;
# run the consumer library's tests that exercise the changed surface.
./grok-check.sh -p xai-grok-pager-bin
./grok-test.sh -p xai-grok-pager

# 3. Deliberate broader or release validation (not the default loop).
source ./grok-dev-env.sh
cargo check --workspace --locked
./grok-test.sh -p xai-grok-shell --run-ignored all
cargo build -p xai-grok-pager-bin --release
```

1.6.2 One helper at a time

Run **one** Cargo helper at a time. `./grok-check.sh`, `./grok-test.sh`, and `./grok-dev-runner.sh` all use `./target-dev`. Cargo serializes access to that directory with its own file lock, so a later command waits briefly while an earlier one finishes. That wait is smaller than recompiling the same dependency graph in a second directory.

Do **not** delete `.cargo` file locks or kill another build to bypass waiting. If you need true parallelism, set a separate worktree-local or temporary `CARGO_TARGET_DIR`:

```
CARGO_TARGET_DIR=/tmp/grok-local-target ./grok-check.sh -p xai-grok-shell
```

Each independent worktree gets its own root-local `./target-dev` by default. Never point multiple active worktrees at the same target directory; that breaks isolation and creates lock contention.

1.6.3 Helper scripts

`./grok-check.sh` sources `./grok-dev-env.sh` and runs `cargo check --locked` with whatever crate or extra flags you pass. Use it for fast focused verification.

`./grok-test.sh` sources `./grok-dev-env.sh` and runs `cargo --locked nextest run`. It applies the repository `nextest` configuration in `.config/nextest.toml` (`retries = 0, 300 s` per-test ceiling, JUnit reports at `target-dev/nextest/default/nextest-results.xml`).

`./grok-dev-runner.sh` sources `./grok-dev-env.sh` and runs the console with the experimental `claudeli-runtime` feature enabled. It stores artifacts in `./target-dev` alongside the other helpers so dependency artifacts are reused where possible. Targets affected by `claudeli-runtime` are still rebuilt because of the feature flag. Use it only when you need that feature; for normal verification prefer `cargo run` with the canonical environment.

`./grok-off-xai-check.sh` is the off-xAI acceptance battery. It runs the built binary with xAI blackholed and asserts that a run with no xAI reachability still works and that every xAI-hosted surface goes quiet under `GROK_XAI_ENABLED=0`. It is not a substitute for `./grok-test.sh`; run it when a change touches an xAI-hosted surface, an endpoint resolver, or the credential identity.

1.6.4 Direct Cargo commands

Direct cargo commands are allowed only when a helper cannot express the needed operation. Before running any direct cargo check, cargo test, cargo clippy, cargo fmt, or similar command from Bash, source `./grok-dev-env.sh` (or reproduce its exact environment) so the command:

- uses the isolated `~/.grokdev` profile;
- sets `CARGO_TARGET_DIR` to `./target-dev` (or honors your explicit override);
- disables Cursor / Claude / Codex state discovery;
- enables sccache when available.

```
source ./grok-dev-env.sh
cargo test -p xai-grok-config
cargo clippy -p xai-grok-shell
cargo fmt --all --check
```

1.6.5 Test runners

cargo-nextest is the fast local default. Nextest runs each test as a separate process and honors `.config/nextest.toml`. cargo test remains fully supported for shared-process semantics and ignores `.config/nextest.toml`. Use nextest for speed by default; use cargo test when the different process model matters.

```
# Fast default: one affected crate.
./grok-test.sh -p xai-grok-shell

# Filter to a test family module or single test. Nextest filters use the
# test module path, so the family module prefix narrows scheduling.
./grok-test.sh -p xai-grok-shell -- session_runtime_family
./grok-test.sh -p xai-grok-shell -- session_runtime_family::test_fork_session

# Include ignored acceptance / perf tests when you have the needed fixtures.
./grok-test.sh -p xai-grok-shell --run-ignored all

# List or dry-run without executing tests.
source ./grok-dev-env.sh
cargo nextest list -p xai-grok-shell
./grok-test.sh -p xai-grok-shell --no-run

# Cargo-test compatibility and quick lints.
source ./grok-dev-env.sh
cargo test -p xai-grok-shell
cargo clippy -p xai-grok-shell
cargo fmt --all --check
```

WARNING: Passing `--no-capture` to installed cargo-nextest disables output capture and serializes test execution. Use it only for short diagnostic runs where live output matters more than speed.

1.6.6 Running the console

For ordinary verification, use the freshly built source checkout and disable leader reuse:

```
source ./grok-dev-env.sh
cargo run -p xai-grok-pager-bin -- --no-leader --no-auto-update
```

For inline terminal output during diagnostics:

```
source ./grok-dev-env.sh
cargo run -p xai-grok-pager-bin -- \
  --no-leader \
  --no-auto-update \
  --no-alt-screen
```

Use the alternate-screen TUI when testing full-screen rendering. Use `--no-alt-screen` only when persistent diagnostic output is more useful.

1.6.7 Interactive TUI verification with tmux

A full-screen TUI cannot be exercised from a non-interactive command shell: it needs a real PTY and a session that persists between commands. The sanctioned pattern is a detached tmux session the agent drives with `send-keys` and observes with `capture-pane`. The enforceable policy (setup, full command set, PNG screenshots, and rules) is in [AGENTS.md](#); the minimum shape is:

```
# Build the binary first so the session runs fresh code.
bash -c 'source ./grok-dev-env.sh && cargo build -p xai-grok-pager-bin'

# Detached session with the canonical environment inline (tmux does not
# inherit shell exports) and the ordinary --no-leader / --no-auto-update flags.
tmux new-session -d -s grok-dev -x 200 -y 50 \
  "export GROK_HOME=\"$HOME/.grokdev\" \
  GROK_LEADER_SOCKET=\"$HOME/.grokdev/leader.sock\" \
  GROK_DISABLE_AUTOUPDATER=1; \
  ./target-dev/debug/xai-grok-pager --no-leader --no-auto-update; exec zsh"

# Observe (text is the default evidence; -S -N includes N lines of
# scrollbar; -e preserves ANSI colors).
tmux capture-pane -p -t grok-dev
tmux capture-pane -p -t grok-dev -S -500
tmux capture-pane -e -p -t grok-dev

# Interact, wait briefly, then capture again.
tmux send-keys -t grok-dev '/tersify lite' Enter
```

Caveats that bite in practice:

- The agent must never `tmux attach`; attaching takes over the user's terminal. All interaction goes through `send-keys` and all observation through `capture-pane`.
- A detached tmux session does not inherit the current shell's exports; the canonical environment must be set inside the session command string.
- The first capture after startup may be blank while the binary initializes; wait briefly and capture again before diagnosing.
- Kill the session when verification ends so a stale console cannot hold the development profile.

1.6.8 Headless mode

```
source ./grok-dev-env.sh
cargo run -p xai-grok-pager-bin -- \
  --no-leader \
  --no-auto-update \
  -p "Describe the current repository."
```

Headless calls can contact configured model services and consume credentials or quota from the development profile. Do not run them unless that external action is relevant to the task.

1.6.9 Makefile (release-dist and deployment only)

The repository Makefile is release-dist and deployment-oriented; it is not for normal development. Its default `make build` target compiles `xai-grok-pager-bin` with the `release-dist` profile, `--locked`, `--timings`, `CARGO_BUILD_JOBS`, `sccache/RUSTC_WRAPPER`, and the optional `FEATURES` Cargo-feature variable.

```
make build # host arch, CPU-tuned (what `make deploy` uses)
make build MACOS_TARGET_CPU=apple-a17 # tune for M1/M2/M3 instead of M4
make build-x64 # Intel slice (x86_64-apple-darwin), opt-in
make build-universal # host + Intel, lipo'd into one artifact, opt-in
make build FEATURES=claude-cli-runtime
```

Local is make deploy, host-arch and CPU-tuned. It builds, signs, and installs the native release-dist artifact for this workstation: on an Apple Silicon host that is the arm64 slice with `-C target-cpu=apple-m4` by default (`MACOS_TARGET_CPU` overrides it, `native` or an empty value are accepted). No cross build is in the local loop.

`apple-m4` is additive over rustc's default for this target — verified with `rustc --print cfg`: it only adds `target_feature="bf16", "bti"` and `"i8mm"` (no SME), so a locally built binary also runs on older Apple Silicon. Use `MACOS_TARGET_CPU=apple-a17` to match the release `m1` variant exactly.

`RUSTFLAGS` replaces the per-target list in `.cargo/config.toml`, so the arm64 base flags are repeated verbatim before the tuning flag; build scripts and proc macros inherit them too, which is safe locally because they run on this same host. Changing `MACOS_TARGET_CPU` (or toggling it off) re-fingerprints the whole graph: expect one full rebuild, then the cache holds.

The Intel slice belongs to the release pipeline. `.github/workflows/ release-build.yml` publishes `grok-x86_64-apple-darwin-intel` (baseline `x86-64`, so pre-AVX2 2010-2012 Macs work; a haswell/skylake tuning would SIGILL on them). Local helpers that mirror it, for parity checks only:

- `make build-x64` — cross-compile it (macOS SDK handles both arches; the target is pinned in `rust-toolchain.toml` and its `rustflags` already live in `.cargo/config.toml`).
- `make deploy UNIVERSAL=1` — deploy one lipo-combined universal binary (CPU-tuned arm64 + baseline x64) instead of the thin host artifact. `make verify` then also asserts both slices are present and executes the Intel slice under Rosetta when available.

Runner pools. Tag builds run on the self-hosted `grok-*` runners (this Mac for macOS, `l-agent-lnx` on psc for Linux, `gha-runner-win` on psc for Windows); branch pushes stay on the GitHub-hosted images, which are ephemeral, parallel, and never occupy the development machines. `workflow_dispatch` accepts runners: `self-hosted|hosted` to force either pool, and the hosted labels stay in the matrix as the fallback. Both crates' build scripts retry the ripgrep release asset three times per base and fall back to the public URL, and `GROK_SHELL_RG_DOWNLOAD_BASE` / `GROK_TOOLS_RG_DOWNLOAD_BASE` (forwarded from the `GROK_RG_DOWNLOAD_BASE` repository variable) can point them at a mirror.

Do not run `make build`, `make deploy`, `make deploy-binary`, `make deploy-wrapper`, or `make verify` in ordinary edit loops. Deployment and verify targets require an explicit user request.

1.6.10 Version identity and bumps (fcustom)

The user-visible version of the fork comes from exactly four **lockstep CLI crates**, whose version fields must always stay identical:

- `crates/codegen/xai-grok-pager/Cargo.toml`
- `crates/codegen/xai-grok-shell/Cargo.toml`
- `crates/codegen/xai-grok-pager-bin/Cargo.toml`
- `crates/codegen/xai-grok-version/Cargo.toml`

The `fcustom` base is `1.0.0` (bumped from the upstream `0.2.110` base in commit `807fc91`; local tag `v1.0.0-fcustom`). Other workspace crates keep their own independent versions.

How the version reaches the binary:

1. The manifest version becomes `CARGO_PKG_VERSION` and `xai_grok_version::VERSION`. These feed `grok --version`, the welcome screen, the `x-grok-client-version` telemetry header, MCP client identification, and internal user-agent strings.
2. `xai-grok-pager/build.rs` bakes `VERSION_WITH_COMMIT` as `<version> (<short commit HEAD>)`, so each build embeds the exact commit.
3. A `GROK_VERSION` environment variable at build time overrides the manifest in both the pager build script and `xai-grok-version`. Use it only for one-off experimental builds; the manifest is the persistent source of truth.
4. The channel suffix `( [stable] / [alpha] )` is derived at runtime by comparing the compiled version against the cached stable pointer in `$GROK_HOME/version.json` (written by the auto-updater). Because the `fcustom` base is ahead of the upstream feed, deployed builds display `[alpha]`. This is expected and harmless.

Bump procedure (only on explicit release intent):

1. Choose **plain semver with no pre-release suffix** (for example 1.0.1, 1.1.0). A pre-release suffix such as 1.0.1-fcustom makes the stable-channel updater force-install the feed's latest stable over the custom build (needs_update in xai-grok-update always returns true for pre-release currents on stable). Non-semver strings are rejected by Cargo in manifests and, if forced via GROK_VERSION, break minimum_version and version_overrides parsing.
2. Edit exactly the four lockstep manifests. No workspace dependency pins these crates by version (all requirements are path-based), so no other manifest changes are needed. Never edit the generated root Cargo.toml.
3. Regenerate Cargo.lock with a direct cargo command in the canonical isolated environment (the helpers pass --locked and refuse the change, so a direct command is the sanctioned path here). Review the diff: it must contain only the four member version lines. Commit the manifests and the lockfile together.
4. Rebuild and redeploy with make deploy, then make verify (explicit user request required, as for all deployment targets). Confirm the deployed binary reports the new version and the new commit hash.
5. Optionally mark the commit with a local annotated tag v<major.minor.patch>-fcustom. Tag pushes are blocked by the repository push policy (fcustom -> fcustom only); tags are local markers.

Expected side effects of a bump: the deployed binary shows [alpha] while ahead of the cached stable pointer, and any running leader from the previous version is evicted on reconnect due to version skew (normal).

Version-selection invariant: keep the fcustom base **ahead of the upstream stable feed** (currently 1.0.0 > 0.2.x). Then target > current is false for every feed release and the auto-updater never replaces a custom build with a feed release.

1.6.11 Cargo.lock discipline

Cargo.lock is tracked and must remain deterministic.

- Use --locked for reproducible CI/release verification and whenever dependency resolution must not change.
- Ordinary source-only changes should not modify Cargo.lock. If git diff shows lock changes after editing code only, the environment is out of sync; investigate before committing.
- When a dependency, feature, or manifest change intentionally alters resolution, regenerate the lockfile with the pinned toolchain in the canonical isolated environment, review the resulting diff, and include it in the change.
- Never hand-edit Cargo.lock.
- Do not run cargo update broadly. For a targeted update, use cargo update -p <name> --precise <version>.
- If --locked fails, investigate the manifest/lock mismatch rather than removing --locked or deleting Cargo.lock.

Distinguish the persistent Cargo.lock dependency lockfile from transient file locks Cargo creates under .cargo and target/.

1.6.12 Tooling policy

Do not add custom build serialization, new linker flags, mold, or alternative caching without measurement and an explicit rationale. Cargo's own file locks, the shared worktree-local ./target-dev, and sccache are the current policy. sccache and Linux lld are already configured in .cargo/config.toml.

1.6.13 Measuring compile/link cost

Install DotSlash before commands that require bin/protoc:

```
cargo install dotslash
/usr/bin/env dotslash --help
```

When changing the test layout, measure the impact before adding custom linker replacements. Rust link times are usually the bottleneck for multi-binary integration-test suites.

```
# cargo-test: compile every test target without executing.
source ./grok-dev-env.sh
cargo test -p xai-grok-shell --no-run --timings
```

```
# nextest: build and resolve test metadata without executing tests.
./grok-test.sh -p xai-grok-shell --no-run

# Inspect shared-cache hit rate.
sccache --show-stats
```

1.7 Testing Strategy

- Add focused unit tests beside changed code.
- Use crate-level integration tests for cross-module or process behavior.
- Use PTY tests for terminal lifecycle, input, rendering, reconnect, and full-screen behavior.
- Use snapshots for stable visual output.
- Test both success and failure paths for permissions, sandboxing, MCP persistence, reconnection, and recovery.
- Avoid tests that depend on a developer's existing configuration, credentials, sessions, shell aliases, or running leader process.
- Use temporary directories for test-specific mutable state whenever possible. The persistent `~/ .grokdev` profile is a safety boundary, not a substitute for hermetic fixtures.

1.8 Change Guidelines

- Use the pinned Rust toolchain and repository formatting configuration.
- Follow standard Rust conventions: four-space indentation, `snake_case` functions and modules, `CamelCase` types and traits, and `SCREAMING_SNAKE_CASE` constants.
- Keep crate boundaries purposeful.
- Check downstream impact before changing common protocol or runtime crates.
- Preserve user-visible behavior unless the task intentionally changes it.
- Keep unrelated working-tree changes intact.
- Never weaken fail-closed managed configuration, permission gates, sandbox restrictions, credential protection, or path containment accidentally.

1.9 Important Starting Points

- `crates/codegen/xai-grok-pager-bin/src/main.rs`: executable composition root.
- `crates/codegen/xai-grok-pager/src/app`: TUI architecture and event loop.
- `crates/codegen/xai-grok-shell/src/agent/app.rs`: agent mode entry points.
- `crates/codegen/xai-grok-shell/src/session`: session runtime.
- `crates/codegen/xai-grok-tools/src/registry`: tool registration.
- `crates/codegen/xai-grok-workspace/src/permission`: permission system.
- `crates/codegen/xai-grok-sandbox/src/lib.rs`: sandbox manager.
- `crates/codegen/xai-grok-config/src/paths.rs`: `GROK_HOME` behavior.
- `crates/codegen/xai-grok-tersify/src/lib.rs`: tersify engine entry point.
- `crates/codegen/xai-grok-shell/src/session/repetition_guard.rs`: repetition-loop guard.
- `crates/common/xai-tool-runtime/src/tool.rs`: tool abstraction.
- `crates/codegen/xai-grok-pager/docs/user-guide`: user documentation.

1.10 Definition of Done

A change is ready for handoff when:

1. the relevant architecture and repository conventions were respected;
2. all authored content uses `en-US`;
3. all commands ran with `GROK_HOME=${HOME}/.grokdev`;
4. no production profile, installed binary, deployed wrapper, or production leader socket was used;
5. focused tests pass;
6. formatting and applicable lints pass;
7. security-sensitive changes include failure-path coverage;
8. interactive TUI behavior was exercised through the `tmux` workflow (see "Interactive TUI verification with `tmux`") when the change affects live TUI interaction;
9. the handoff lists the exact validation commands and their results.